

claude-windows / 02b9e9c9-3806-45ca-9b24-2ecc35a81efd

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\02b9e9c9-3806-45ca-9b24-2ecc35a81efd\subagents\workflows\wf_27df53c8-8e0\agent-aafa2be60a4395ecb.jsonl`  
- SHA-256: `c32a22cc9a2e89c157bbd1d21ec47c9d5dcffca4f0101f6d0171ab40e3f4b22b`  
- Source modified: `2026-06-10T16:30:07+00:00`  
- Imported at: `2026-07-05T16:48:21+00:00`  
- project: `wf_27df53c8-8e0`  
- session_id: `02b9e9c9-3806-45ca-9b24-2ecc35a81efd`
```

Transcript

1. user (2026-06-10T16:22:08.062Z)

You are an adversarial verifier. A code reviewer audited the repo at C:/Users/avidu/Projects/muneem and reported this finding:

```
TITLE: `db backup` deletes the destination before any validation ? `muneem db backup <ledger-path>` destroys the ledger  
SEVERITY CLAIMED: high  
FILE: src/muneem/db.py (line 292-295)  
DESCRIPTION: `backup_database` unlinks `dest` BEFORE opening the source and never checks that dest != source. If the user passes the ledger's own path as the destination (an easy slip ? `db backup` takes the destination as its positional argument, and a user may think they are naming the source), the live encrypted ledger is deleted; `_open_encrypted` then creates a brand-new empty encrypted DB at that path (key verification trivially passes on an empty DB), and the 'backup' completes successfully, leaving an empty ledger. Total silent data loss with exit code 0. There is also no overwrite confirmation for an existing backup file (cli.py:450-465 has no --force / prompt).  
EVIDENCE QUOTED: dest_path = Path(dest)  
    dest_path.parent.mkdir(parents=True, exist_ok=True)  
    dest_path.unlink(missing_ok=True)  
    source = _open_encrypted(str(db_path), key) # validates key  
SUGGESTED FIX: Resolve both paths and refuse if `dest_path.resolve() == Path(db_path).resolve()` (also reject dest equal to the source's -wal/-shm siblings). Open and validate the source BEFORE unlinking dest. In the CLI, require confirmation (or --force) when dest already exists.
```

Your job is to try to REFUTE it. Your lens: correctness: trace the actual code path the finding describes and check the claimed behavior is what the code really does (read the file and its callers/tests).

Read the actual code in the repo (and tests/config where relevant). If after reading you cannot confirm the problem is real, set isReal=false. If it is real but over/under-stated, set adjustedSeverity accordingly. Be strict: plausible-sounding but unconfirmed = refuted.

SENSITIVE-DATA RULES (mandatory, highest priority):

```
- testdata/aubank-statements/, testdata/axis-statements/, testdata/hdfc-statements/, testdata/snapshots/, and testdata/credentials/ in the muneem repo contain the user's REAL financial documents and a real OAuth client file. Do NOT open/read the PDFs. You MAY check file existence and git-tracking status (git ls-files, git log --all -- <path>).  
- If you must inspect testdata/snapshots/*.json or testdata/credentials/*.json to assess sensitivity, read at most the JSON key structure and NEVER quote values (no account numbers, balances, person names, client secrets, transaction amounts) in your output.  
- Never include sensitive values in your findings. Refer to them abstractly ("a real account number", "a client_secret value").  
- Do not use WebSearch/WebFetch and do not send any repo content to any external service.  
Structured output only.
```

2. user (2026-06-10T16:22:13.960Z)

```

240
241         # Verify the encrypted copy BEFORE the destructive swap: it must open with the
key, pass
242         # an integrity check, and carry every table across. (The plaintext .bak is kept
either way,
243         # so a failed check is recoverable ? but we refuse to swap in a bad file.)
244         with connect(str(enc_tmp), key=key) as check:
245             integrity = check.execute("PRAGMA integrity_check").fetchone()[0]
246             ntables = check.execute(
247                 "SELECT count(*) FROM sqlite_master WHERE type = 'table'"
248             ).fetchone()[0]
249         if integrity != "ok" or ntables != src_tables:
250             raise EncryptedDatabaseError(
251                 f"encryption migration verification failed for {path.name} "
252                 f"(integrity={integrity!r}, tables {ntables}/{src_tables}); original
left untouched"
253             )
254
255         backup = path.with_name(path.name + ".plaintext.bak")
256         backup.unlink(missing_ok=True)
257         path.rename(backup)
258         enc_tmp.rename(path) # consumed on success
259         _harden_permissions(str(path))
260     finally:
261         # Never leave a partial temp behind: no-op on success (already renamed), cleans
up on error.
262         enc_tmp.unlink(missing_ok=True)
263     return int(ntables)
264
265
266     def rekey_database(db_path: Path | str, current_key: str, new_key: str) -> None:
267         """Re-encrypt the ledger under ``new_key`` in place (SQLCipher ``PRAGMA rekey``).
268
269         Opens with ``current_key`` first (which verifies it), then rekeys. Raises
270         :class:`EncryptedDatabaseError` if ``current_key`` is wrong or the file is not an
encrypted
271         muneem ledger. The caller is responsible for updating the keychain entry to
``new_key`` only
272         after this returns successfully.
273         """
274         conn = _open_encrypted(str(db_path), current_key) # validates current_key
275         try:
276             conn.execute(f"PRAGMA rekey = \"x'{new_key}'\"")
277             conn.commit()
278         finally:
279             conn.close()
280         _harden_permissions(str(db_path))
281
282
283     def backup_database(db_path: Path | str, dest: Path | str, key: str) -> None:
284         """Write an **encrypted** online-backup copy of the ledger to ``dest`` (same key).
285
286         Uses the SQLite Online Backup API for a consistent snapshot ? safe even with WAL
active, so no
287         naive file copy and no manual checkpoint needed. The destination is keyed with the
same key, so
288         it stays encrypted at rest. Raises :class:`EncryptedDatabaseError` if ``key`` is
wrong.
289         """
290         import sqlcipher3
291
292         dest_path = Path(dest)
293         dest_path.parent.mkdir(parents=True, exist_ok=True)
294         dest_path.unlink(missing_ok=True)
295         source = _open_encrypted(str(db_path), key) # validates key

```

```

296         try:
297             target = sqlcipher3.connect(str(dest_path))
298         try:
299             target.execute(_raw_key_pragma(key)) # key the destination so the backup is
encrypted
300             source.backup(target) # consistent snapshot of the live DB
301         finally:
302             target.close()
303     finally:
304         source.close()
305     _harden_permissions(str(dest_path))
306
307
308     def init_db(conn: sqlite3.Connection) -> None:
309         """Bring the schema up to date via the migration runner (creates tables on a fresh
DB).
310
311         The current ``_SCHEMA`` is the frozen *baseline* (version ``SCHEMA_VERSION``);
future changes
312         are forward migrations in :mod:`muneem.migrations`, recorded in
``schema_migrations``.
313         """
314         from muneem import migrations
315
316         migrations.run_migrations(conn, baseline_sql=_SCHEMA,
baseline_version=SCHEMA_VERSION)
317
318
319     def get_document_by_sha(conn: sqlite3.Connection, sha256: str) -> sqlite3.Row | None:
320         return conn.execute("SELECT * FROM documents WHERE sha256 = ?",
(sha256,)).fetchone()
321
322
323     def _insert_row(conn: sqlite3.Connection, table: str, fields: dict) -> int:
324         """Internal helper for the simple INSERT pattern used by documents, offers, and bank
txns."""
325         cols = ", ".join(fields)
326         placeholders = ", ".join("?" for _ in fields)
327         cur = conn.execute(
328             f"INSERT INTO {table} ({cols}) VALUES ({placeholders})",
329             tuple(fields.values()),
330         )
331         rowid = cur.lastrowid # always set after a successful INSERT; guard for the type-
checker
332         if rowid is None: # pragma: no cover - defensive; INSERT always sets lastrowid
333             raise RuntimeError(f"INSERT into {table} did not return a row id")
334         return int(rowid)
335
336
337     def insert_document(conn: sqlite3.Connection, **fields: object) -> int:
338         """Insert into documents table. Delegates to the shared helper for DRY."""
339         return _insert_row(conn, "documents", dict(fields))
340
341
342     def record_provenance(
343         conn: sqlite3.Connection,
344         *,
345         document_id: int,
346         source_type: str,
347         parser: str | None = None,
348         parser_version: str | None = None,
349         mapping_layer: str | None = None,

```

```

420         raise typer.Exit(1)
421     key = peek_key()
422     if not key:
423         typer.echo("no encryption key in the keychain for this ledger.", err=True)
424         raise typer.Exit(1)
425     return key
426
427
428 @db_app.command("rotate-key")
429 def db_rotate_key(
430     db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
431 dir)."),
432 ) -> None:
433     """Rotate the ledger's encryption key (SQLCipher rekey) and update the keychain."""
434     from muneem import db as dbmod
435     from muneem import db_key
436
437     db_path = db or load_config().db_path
438     current = _require_encrypted_ledger(db_path)
439     fresh = db_key.new_key()
440     # Keychain-first with rollback: if the file rekey fails, the keychain still matches
441     the file.
442     db_key.set_key(fresh)
443     try:
444         dbmod.rekey_database(db_path, current, fresh)
445     except dbmod.EncryptedDatabaseError as exc:
446         db_key.set_key(current) # roll back; the file is still under the old key
447         typer.echo(str(exc), err=True)
448         raise typer.Exit(1) from exc
449     typer.echo(f"rotated the encryption key for {Path(db_path).name}.")
450
451
452 @db_app.command("backup")
453 def db_backup(
454     dest: Path = typer.Argument(..., help="Destination path for the encrypted backup
455 copy."),
456     db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
457 dir)."),
458 ) -> None:
459     """Write an encrypted online-backup copy of the ledger (consistent even with WAL
460 active)."""
461     from muneem import db as dbmod
462
463     db_path = db or load_config().db_path
464     key = _require_encrypted_ledger(db_path)
465     try:
466         dbmod.backup_database(db_path, dest, key)
467     except dbmod.EncryptedDatabaseError as exc:
468         typer.echo(str(exc), err=True)
469         raise typer.Exit(1) from exc
470     typer.echo(f"backed up {Path(db_path).name} to {dest} (encrypted).")
471
472
473 @db_app.command("setup-recovery")
474 def db_setup_recovery(
475     out: Path | None = typer.Option(
476         None, help="Recovery file path (default: <data dir>/muneem-recovery.json).")
477 ),
478 ) -> None:
479     """Create a passphrase-wrapped recovery file for the ledger's encryption key.
480
481     Lets you restore the key (and thus the ledger + backups) if the OS keychain is ever
482     lost. The
483     file holds only the *wrapped* key ? useless without the passphrase.
484 """

```

```

479         import getpass
480
481         from muneem import recovery
482         from muneem.db_key import peek_key
483
484         key = peek_key()
485         if not key:
486             typer.echo("no encryption key in the keychain yet (create the ledger first).",
err=True)
487             raise typer.Exit(1)
488         out_path = out or (load_config().data_dir / "muneem-recovery.json")
489         passphrase = getpass.getpass("New recovery passphrase: ")
490         if passphrase != getpass.getpass("Confirm passphrase: "):
491             typer.echo("passphrases did not match.", err=True)
492             raise typer.Exit(1)
493         try:
494             recovery.write_recovery_file(out_path, key, passphrase)
495         except recovery.RecoveryError as exc:
496             typer.echo(str(exc), err=True)
497             raise typer.Exit(1) from exc
498         typer.echo(
499             f"wrote recovery file to {out_path}. Store it OFF this machine (password manager
/ "

```

4. user (2026-06-10T16:22:29.738Z)

```

1         """Local SQLite store: the ``documents`` and ``offers`` slice of the schema.
2
3         This is the minimal slice the promo spine needs (ROADMAP A.2). The full schema
4         (accounts, transactions, holdings, merchants, items) grows with later milestones.
5         ``documents.sha256`` is UNIQUE so re-ingesting the same file is a no-op.
6         ``offers.source`` records provenance ('text' = exact text layer, 'ocr' = approximate).
7         """
8
9         from __future__ import annotations
10
11         import os
12         import sqlite3
13         from collections.abc import Iterator
14         from contextlib import contextmanager
15         from pathlib import Path
16
17         from muneem.errors import EncryptedDatabaseError # re-exported so
db.EncryptedDatabaseError works
18
19         SCHEMA_VERSION = 4
20
21         # Unencrypted SQLite files start with this 16-byte magic header; an encrypted
(SQLCipher)
22         # file does not, so we can tell them apart without a key.
23         _SQLITE_MAGIC = b"SQLite format 3\x00"
24
25         _SCHEMA = """
26         CREATE TABLE IF NOT EXISTS documents (
27             id            INTEGER PRIMARY KEY,
28             source         TEXT NOT NULL,
29             external_id    TEXT,
30             sender         TEXT,
31             subject        TEXT,
32             received_date  TEXT,
33             doc_type       TEXT,
34             sha256         TEXT NOT NULL UNIQUE,
35             path           TEXT,
36             status         TEXT NOT NULL DEFAULT 'parsed',
37             created_at     TEXT NOT NULL DEFAULT (datetime('now'))

```

```

38 );
39
40 CREATE TABLE IF NOT EXISTS offers (
41     id INTEGER PRIMARY KEY,
42     document_id INTEGER NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
43     merchant TEXT,
44     promo_code TEXT,
45     description TEXT,
46     expiry_raw TEXT,
47     expiry_type TEXT,
48     source TEXT NOT NULL DEFAULT 'text',
49     redeemed INTEGER NOT NULL DEFAULT 0,
50     source_text TEXT
51 );
52
53 CREATE INDEX IF NOT EXISTS idx_offers_document ON offers(document_id);
54
55 -- We use separate tables for each bank (e.g. axis_transactions, hdfc_transactions)
56 -- instead of a unified table to preserve raw schema fidelity of the source PDFs.
57 -- Normalization into a unified view layer is planned for a later phase
58 -- once all bank schemas are stabilized.
59 CREATE TABLE IF NOT EXISTS axis_transactions (
60     id INTEGER PRIMARY KEY,
61     document_id INTEGER NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
62     account_number TEXT,
63     txn_date TEXT,
64     value_date TEXT,
65     particulars TEXT,
66     debit REAL,
67     credit REAL,
68     balance REAL
69 );
70
71 CREATE TABLE IF NOT EXISTS axis_cc_transactions (
72     id INTEGER PRIMARY KEY,
73     document_id INTEGER NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
74     account_number TEXT,
75     txn_date TEXT,
76     particulars TEXT,
77     merchant_category TEXT,
78     amount REAL,
79     type TEXT
80 );
81
82 CREATE TABLE IF NOT EXISTS au_transactions (
83     id INTEGER PRIMARY KEY,
84     document_id INTEGER NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
85     account_number TEXT,
86     txn_date TEXT,
87     value_date TEXT,
88     particulars TEXT,
89     debit REAL,
90     credit REAL,
91     balance REAL
92 );
93
94 CREATE TABLE IF NOT EXISTS hdfc_transactions (
95     id INTEGER PRIMARY KEY,
96     document_id INTEGER NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
97     account_number TEXT,
98     txn_date TEXT,
99     value_date TEXT,
100     particulars TEXT,
101     ref TEXT,
102     withdrawal REAL,

```

```

103         deposit REAL,
104         closing_balance REAL
105     );
106     """
107
108
109     def _raw_key_pragma(key: str) -> str:
110         """The SQLCipher raw-key PRAGMA: use the 256-bit ``key`` (64 hex chars) directly, no
KDF."""
111         return f"PRAGMA key = \"x'{key}'\""
112
113
114     def is_plaintext_sqlite(db_path: Path | str) -> bool:
115         """True iff the file exists and begins with the *unencrypted* SQLite magic header.
116
117         Lets us refuse to silently treat a plaintext ledger as encrypted (and vice-versa)
118         without needing the key.
119         """
120         try:
121             with open(db_path, "rb") as fh:
122                 return fh.read(16) == _SQLITE_MAGIC
123         except OSError:
124             return False
125
126
127     def _harden_permissions(path: str) -> None:
128         """Best-effort: restrict the DB file to its owner (0600). No-op on failure or
``:memory:``."""
129         if path == ":memory:":
130             return
131         try:
132             os.chmod(path, 0o600)
133         except OSError:
134             pass
135
136
137     def _open_encrypted(path: str, key: str) -> sqlite3.Connection:
138         """Open an encrypted connection via SQLCipher, verifying the key before returning.
139
140         A keyed open NEVER silently downgrades to plaintext: if the binding is missing, or
the
141         key is wrong (or the file is not an encrypted muneem ledger), this raises.
142         """
143         try:
144             import sqlcipher3
145         except ImportError as exc: # pragma: no cover - exercised only without the binding
146             raise EncryptedDatabaseError(
147                 "database encryption requested but the 'sqlcipher3' package is not
installed"
148             ) from exc
149
150         conn = sqlcipher3.connect(path)
151         conn.execute(_raw_key_pragma(key)) # MUST be the first statement on the connection
152         try:
153             conn.execute("SELECT count(*) FROM sqlite_master") # forces key verification
154         except sqlcipher3.DatabaseError as exc:
155             conn.close()
156             raise EncryptedDatabaseError(
157                 f"could not open encrypted database {Path(path).name}: wrong key, or not an
"
158                 "encrypted muneem ledger"
159             ) from exc
160         conn.row_factory = sqlcipher3.Row
161         return conn
162

```

```

163
164     @contextmanager
165     def connect(db_path: Path | str, *, key: str | None = None) ->
Iterator[sqlite3.Connection]:
166         """Open a connection (commit on success, always close). Parent dirs created.
167
168         With ``key`` (a 64-hex-char SQLCipher key) the database is **encrypted at rest** via
169         SQLCipher. Without it, a plaintext stdlib :mod:`sqlite3` connection is used ? the
path for
170         ``:memory:`` and tests. A keyed open never silently falls back to plaintext (see
171         :func:`_open_encrypted`); the production ledger is opened via
:func:`connect_encrypted`.
172         """
173         path = str(db_path)
174         if path != ":memory:":
175             Path(path).parent.mkdir(parents=True, exist_ok=True)
176         if key is not None:
177             conn = _open_encrypted(path, key)
178             _harden_permissions(path)
179         else:
180             conn = sqlite3.connect(path)
181             conn.row_factory = sqlite3.Row
182             conn.execute("PRAGMA foreign_keys = ON")
183         try:
184             yield conn
185             conn.commit()
186         finally:
187             conn.close()
188
189
190     @contextmanager
191     def connect_encrypted(db_path: Path | str) -> Iterator[sqlite3.Connection]:
192         """Open the real on-disk ledger, encrypted with the keychain-managed key.
193
194         The production path: resolves (or mints, on first use) the 256-bit key from the OS
195         keychain and opens with SQLCipher. If ``db_path`` is a pre-existing *plaintext*
SQLite
196         file, refuses with a clear pointer to ``muneem db encrypt`` rather than a cryptic
197         "file is not a database" error.
198         """
199         from muneem.db_key import get_or_create_key
200
201         path = str(db_path)
202         if is_plaintext_sqlite(path):
203             raise EncryptedDatabaseError(
204                 f"{Path(path).name} is an unencrypted SQLite database; run 'muneem db
encrypt' "
205                 "to migrate it to an encrypted ledger"
206             )
207         with connect(path, key=get_or_create_key()) as conn:
208             yield conn
209
210
211     def encrypt_database(db_path: Path | str, key: str) -> int:
212         """Migrate a plaintext SQLite ledger to an encrypted one in place, via
``sqlcipher_export``.
213
214         The original is preserved as ``<name>.plaintext.bak`` for the caller to verify and
then
215         securely delete (it still holds the cleartext data). Returns the number of user
tables
216         copied. Raises :class:`EncryptedDatabaseError` if ``db_path`` is not a plaintext
SQLite file.
217         """
218         import sqlcipher3

```

```

219
220     path = Path(db_path)
221     if not is_plaintext_sqlite(path):
222         raise EncryptedDatabaseError(
223             f"{path.name} is not a plaintext SQLite database (already encrypted or
missing)"
224         )
225
226     enc_tmp = path.with_name(path.name + ".enc.tmp")
227     enc_tmp.unlink(missing_ok=True)
228     try:
229         src = sqlcipher3.connect(str(path)) # open the plaintext source (no key)
230         try:
231             src_tables = src.execute(
232                 "SELECT count(*) FROM sqlite_master WHERE type = 'table'"
233             ).fetchone()[0]
234             target = str(enc_tmp).replace("\\", "/") # SQL string literal: forward
slashes
235             src.execute(f"ATTACH DATABASE '{target}' AS encrypted KEY '\\x'{key}'\\'")
236             src.execute("SELECT sqlcipher_export('encrypted')")
237             src.execute("DETACH DATABASE encrypted")
238         finally:
239             src.close()

```

5. user (2026-06-10T16:23:12.761Z)

```

390     ) -> None:
391         """Migrate an existing *plaintext* ledger to an encrypted one (SQLCipher), in
place."""
392         from muneem import db as dbmod
393         from muneem.db_key import get_or_create_key
394
395         db_path = db or load_config().db_path
396         if not Path(db_path).exists():
397             typer.echo(
398                 f"nothing to encrypt: {db_path} does not exist (a new ledger is born
encrypted)."
399             )
400             return
401         if not dbmod.is_plaintext_sqlite(db_path):
402             typer.echo(
403                 f"{Path(db_path).name} is already encrypted (or not a SQLite file); nothing
to do."
404             )
405             return
406         n = dbmod.encrypt_database(db_path, get_or_create_key())
407         typer.echo(
408             f"encrypted {Path(db_path).name}: {n} table(s) migrated. A plaintext backup
remains at "
409             f"{Path(db_path).name}.plaintext.bak ? verify, then securely delete it."
410         )
411
412
413     def _require_encrypted_ledger(db_path: Path) -> str:
414         """Return the ledger's keychain key, or exit with a clear message if it can't be
used."""
415         from muneem import db as dbmod
416         from muneem.db_key import peek_key
417
418         if not Path(db_path).exists() or dbmod.is_plaintext_sqlite(db_path):
419             typer.echo("no encrypted ledger here (run 'muneem db encrypt' first).",
err=True)
420             raise typer.Exit(1)
421         key = peek_key()

```

6. user (2026-06-10T16:23:13.826Z)

```
tests\test_db_crypto_ops.py:1: """Crypto operations on the encrypted ledger: key rotation,
encrypted backup, migration temp
tests\test_db_crypto_ops.py:52:# --- encrypted backup
-----
tests\test_db_crypto_ops.py:55: def test_backup_is_encrypted_and_intact(tmp_path):
tests\test_db_crypto_ops.py:58:     dest = tmp_path / "backup" / "led.bak.db"
tests\test_db_crypto_ops.py:59:     dbmod.backup_database(p, dest, KEY_A)
tests\test_db_crypto_ops.py:60:     assert dest.exists() and not dbmod.is_plaintext_sqlite(dest)
# backup stays encrypted
tests\test_db_crypto_ops.py:63:     with pytest.raises(dbmod.EncryptedDatabaseError): # wrong
key can't read the backup
tests\test_db_crypto_ops.py:68: def test_backup_with_wrong_key_raises(tmp_path):
tests\test_db_crypto_ops.py:72:     dbmod.backup_database(p, tmp_path / "b.db", KEY_B)
tests\test_db_crypto_ops.py:118:# --- CLI: db rotate-key / db backup
-----
tests\test_db_crypto_ops.py:142: def test_db_backup_cli(tmp_path):
tests\test_db_crypto_ops.py:145:     dest = tmp_path / "led.backup.db"
tests\test_db_crypto_ops.py:146:     res = runner.invoke(app, ["db", "backup", str(dest), "--db",
str(led)])
tests\test_precommit_hook.py:39:         "data/muneem.db.plaintext.bak", # cleartext backup from
the encrypt migration
tests\test_precommit_hook.py:41:         "backups/my-recovery.json",
```

7. assistant (2026-06-10T16:23:20.151Z)

No existing test covers `dest == source`. Now I'll reproduce the exact scenario empirically with a throwaway encrypted DB (synthetic data only).

8. user (2026-06-10T16:23:21.329Z)

.venv\Scripts\python.exe

9. user (2026-06-10T16:23:21.830Z)

```
1      """Crypto operations on the encrypted ledger: key rotation, encrypted backup, migration
temp
2      cleanup, correctness at size, and the plaintext-refusal guidance ? plus their CLI
commands.
3
4      Offline, fabricated data only; the autouse conf test fixture keeps the real keychain
untouched.
5      """
6
7      import sqlite3
8
9      import pytest
10
11     pytest.importorskip("sqlcipher3")
12
13     from typer.testing import CliRunner # noqa: E402
14
15     from muneem import db as dbmod # noqa: E402
16     from muneem import db_key # noqa: E402
17     from muneem.cli import app # noqa: E402
18
19     runner = CliRunner()
20
21     KEY_A = "00112233445566778899aabbccddeeff00112233445566778899aabbccddeeff"
22     KEY_B = "ffeeddccbbbaa99887766554433221100ffeeddccbbbaa99887766554433221100"
23
24
25     def _seed_encrypted(path, key, n=3):
```

```

26         with dbmod.connect(path, key=key) as conn:
27             conn.execute("CREATE TABLE t (id INTEGER PRIMARY KEY, amount REAL)")
28             conn.executemany("INSERT INTO t (amount) VALUES (?)", [(float(i),) for i in
range(n)])
29
30
31     # --- rekey / rotation
-----
32
33
34     def test_rekey_changes_the_key_and_keeps_data(tmp_path):
35         p = tmp_path / "led.db"
36         _seed_encrypted(p, KEY_A, n=5)
37         dbmod.rekey_database(p, KEY_A, KEY_B)
38         with pytest.raises(dbmod.EncryptedDatabaseError): # old key no longer opens it
39             with dbmod.connect(p, key=KEY_A):
40                 pass
41         with dbmod.connect(p, key=KEY_B) as conn: # new key works, data intact
42             assert conn.execute("SELECT count(*) FROM t").fetchone()[0] == 5
43
44
45     def test_rekey_with_wrong_current_key_raises(tmp_path):
46         p = tmp_path / "led.db"
47         _seed_encrypted(p, KEY_A)
48         with pytest.raises(dbmod.EncryptedDatabaseError):
49             dbmod.rekey_database(p, KEY_B, KEY_A)
50
51
52     # --- encrypted backup
-----
53
54
55     def test_backup_is_encrypted_and_intact(tmp_path):
56         p = tmp_path / "led.db"
57         _seed_encrypted(p, KEY_A, n=4)
58         dest = tmp_path / "backup" / "led.bak.db"
59         dbmod.backup_database(p, dest, KEY_A)
60         assert dest.exists() and not dbmod.is_plaintext_sqlite(dest) # backup stays
encrypted
61         with dbmod.connect(dest, key=KEY_A) as conn:
62             assert conn.execute("SELECT count(*) FROM t").fetchone()[0] == 4
63         with pytest.raises(dbmod.EncryptedDatabaseError): # wrong key can't read the backup
64             with dbmod.connect(dest, key=KEY_B):
65                 pass
66
67
68     def test_backup_with_wrong_key_raises(tmp_path):
69         p = tmp_path / "led.db"
70         _seed_encrypted(p, KEY_A)
71         with pytest.raises(dbmod.EncryptedDatabaseError):
72             dbmod.backup_database(p, tmp_path / "b.db", KEY_B)
73
74
75     # --- migration cleanup + correctness at size + plaintext refusal
-----

```

10. user (2026-06-10T16:25:33.897Z)

Command running in background with ID: bfoo51v4e. Output is being written to:
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-muneem\02b9e9c9-3806-45ca-9b24-2ecc35a81efd\tasks\bfoo51v4e.output. You will be notified when it
completes. To check interim output, use Read on that file path.

11. user (2026-06-10T16:25:47.857Z)

<system-reminder>Warning: the file exists but is shorter than the provided offset (1). The file has 1 lines.</system-reminder>

12. user (2026-06-10T16:26:08.871Z)

<system-reminder>Warning: the file exists but is shorter than the provided offset (1). The file has 1 lines.</system-reminder>

13. user (2026-06-10T16:26:24.930Z)

Command running in background with ID: ba7af3f92. Output is being written to:
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-muneem\02b9e9c9-3806-45ca-9b24-2ecc35a81efd\tasks\ba7af3f92.output. You will be notified when it completes. To check interim output, use Read on that file path.

14. user (2026-06-10T16:26:41.103Z)

<system-reminder>Warning: the file exists but is shorter than the provided offset (1). The file has 1 lines.</system-reminder>

15. user (2026-06-10T16:26:55.117Z)

```
src\muneem\db.py:194:    The production path: resolves (or mints, on first use) the 256-bit key
                        from the OS
src\muneem\cli.py:81:    """Show resolved local paths (no secrets, no sensitive data)."""
src\muneem\cli.py:96:    since the text needed to pick a parser is itself encrypted. The
                        resolved password is forwarded
src\muneem\cli.py:102:    from muneem.statement_unlock import resolve_password
src\muneem\cli.py:107:    password = resolve_password(pdf, interactive=interactive)
src\muneem\cli.py:113:    ex = extract(pdf, password=password or "") # resolved above; `or ""`
                        keeps the type str
src\muneem\cli.py:131:    path=str(pdf.resolve()),
src\muneem\cli.py:154:    from muneem.statement_unlock import resolve_password
src\muneem\cli.py:159:    password = resolve_password(pdf, interactive=interactive) or ""
src\muneem\cli.py:284:    path=str(pdf.resolve()),
src\muneem\ocr.py:34:def _resolve_tesseract() -> str | None:
src\muneem\ocr.py:49:    return _resolve_tesseract() is not None
src\muneem\ocr.py:54:    cmd = _resolve_tesseract()
src\muneem\password_rules.py:71:    The ingest path resolves the password *before* it knows the
                        issuer (it needs the decrypted
src\muneem\statement_unlock.py:59:def resolve_password(pdf_path: Path | str, *, interactive:
bool = True) -> str | None:
src\muneem\statement_unlock.py:63:    no candidate (or prompt) unlocked it. A prompt-resolved
                        password is cached for next time.
```

16. assistant (2026-06-10T16:27:01.251Z)

No path-equality guard anywhere. Let me check the repro output again.

17. user (2026-06-10T16:27:03.102Z)

<system-reminder>Warning: the file exists but is shorter than the provided offset (1). The file has 1 lines.</system-reminder>

18. user (2026-06-10T16:27:12.255Z)

Monitor started (task bcopy0qm8j, timeout 200000ms). You will be notified on each event. Keep working ? do not poll or sleep. Events may arrive while you are waiting for the user ? an event is not their reply.

19. assistant (2026-06-10T16:27:33.883Z)

```
import sys, tempfile, threading, os, faulthandler
```

```

sys.path.insert(0, r"C:\Users\avidu\Projects\muneeem\src")
from pathlib import Path
from muneem import db as dbmod

faulthandler.dump_traceback_later(25, exit=True) # watchdog: dump stacks and hard-exit if hung

KEY = "00112233445566778899aabbccddeeff00112233445566778899aabbccddeeff"
tmp = Path(tempfile.mkdtemp())
p = tmp / "led.db"

with dbmod.connect(p, key=KEY) as conn:
    conn.execute("CREATE TABLE t (id INTEGER PRIMARY KEY, amount REAL)")
    conn.executemany("INSERT INTO t (amount) VALUES (?)", [(float(i),) for i in range(5)])

print("before: size =", p.stat().st_size, flush=True)
with dbmod.connect(p, key=KEY) as conn:
    print("before: rows =", conn.execute("SELECT count(*) FROM t").fetchone()[0], flush=True)

print("calling backup_database(p, p, KEY) ...", flush=True)
try:
    dbmod.backup_database(p, p, KEY)
    print("RESULT: backup_database returned WITHOUT error (CLI would exit 0)", flush=True)
except Exception as e:
    print("RESULT: backup_database RAISED:", type(e).__name__, "-", e, flush=True)

print("after: exists =", p.exists(), "size =", (p.stat().st_size if p.exists() else None),
flush=True)
try:
    with dbmod.connect(p, key=KEY) as conn:
        tables = [t[0] for t in conn.execute("SELECT name FROM sqlite_master WHERE
type='table'").fetchall()]
        print("after: tables =", tables, flush=True)
        if "t" in tables:
            print("after: rows =", conn.execute("SELECT count(*) FROM t").fetchone()[0],
flush=True)
        else:
            print("after: TABLE t IS GONE ? DATA LOST", flush=True)
except Exception as e:
    print("after: reopen failed:", type(e).__name__, "-", e, flush=True)

```

```

range(5)])\n\nprint("\nbefore: size =", p.stat().st_size)\nwith dbmod.connect(p, key=KEY) as
conn:\n    print("\nbefore: rows =", conn.execute("\nSELECT count(*) FROM
t\n").fetchone()[0])\n\n# The slip: dest == the ledger itself\ntry:\n    dbmod.backup_database(p,
p, KEY)\n    print("\nbackup_database returned WITHOUT error (would be exit code 0)\n")\nexcept
Exception as e:\n    print("\nbackup_database RAISED:", type(e).__name__, "\n-\n",
e)\n\nprint("\nafter: exists =", p.exists(), "\nsize =", p.stat().st_size if p.exists() else
None)\ntry:\n    with dbmod.connect(p, key=KEY) as conn:\n        tables = conn.execute("\nSELECT
name FROM sqlite_master WHERE type='table'\n").fetchall()\n        print("\nafter: tables =",
[t[0] for t in tables])\n        if any(t[0] == "\nt\n" for t in tables):\n
print("\nafter: rows =", conn.execute("\nSELECT count(*) FROM t\n").fetchone()[0])\nexcept
Exception as e:\n    print("\nreopen failed:", type(e).__name__, "\n-\n", e)\n'\n@$code | &
C:\\Users\\avidu\\Projects\\muneem\\.venv\\Scripts\\python.exe
-)", "task_id": "bfoo51v4e", "task_type": "local_bash", "command": "$code = @\n\nimport tempfile, os,
sys, traceback\nsys.path.insert(0, r"C:\\Users\\avidu\\Projects\\muneem\\src")\n\nfrom pathlib
import Path\n\nfrom muneem import db as dbmod\n\nKEY =
"\n00112233445566778899aabbccddeeff00112233445566778899aabbccddeeff"\n\ntmp =
Path(tempfile.mkdtemp())\n\np = tmp / "\nled.db"\n\nwith dbmod.connect(p, key=KEY) as conn:\n
conn.execute("\nCREATE TABLE t (id INTEGER PRIMARY KEY, amount REAL)\n")\n
conn.executemany("\nINSERT INTO t (amount) VALUES (?)\n", [(float(i),) for i in
range(5)])\n\nprint("\nbefore: size =", p.stat().st_size)\nwith dbmod.connect(p, key=KEY) as
conn:\n    print("\nbefore: rows =", conn.execute("\nSELECT count(*) FROM
t\n").fetchone()[0])\n\n# The slip: dest == the ledger itself\ntry:\n    dbmod.backup_database(p,
p, KEY)\n    print("\nbackup_database returned WITHOUT error (would be exit code 0)\n")\nexcept
Exception as e:\n    print("\nbackup_database RAISED:", type(e).__name__, "\n-\n",
e)\n\nprint("\nafter: exists =", p.exists(), "\nsize =", p.stat().st_size if p.exists() else
None)\ntry:\n    with dbmod.connect(p, key=KEY) as conn:\n        tables = conn.execute("\nSELECT
name FROM sqlite_master WHERE type='table'\n").fetchall()\n        print("\nafter: tables =",
[t[0] for t in tables])\n        if any(t[0] == "\nt\n" for t in tables):\n
print("\nafter: rows =", conn.execute("\nSELECT count(*) FROM t\n").fetchone()[0])\nexcept
Exception as e:\n    print("\nreopen failed:", type(e).__name__, "\n-\n", e)\n'\n@$code | &
C:\\Users\\avidu\\Projects\\muneem\\.venv\\Scripts\\python.exe -"}

```

24. user (2026-06-10T16:27:51.171Z)

```

{"message": "Successfully stopped task: bcpy0qm8j (f=\"/c/Users/avidu/AppData/Local/Temp/claude/C
--Users-avidu-Projects-muneem/02b9e9c9-3806-45ca-9b24-2ecc35a81efd/tasks/bfoo51v4e.output\"; for
i in $(seq 1 90); do s=$(wc -c < \"$f\" 2>/dev/null || echo 0); if [ \"$s\" -gt 5 ]; then cat
\"$f\"; exit 0; fi; sleep 2; done; echo \"TIMEOUT: repro script produced no output after 180s
(likely hung)\")", "task_id": "bcpy0qm8j", "task_type": "local_bash", "command": "f=\"/c/Users/avidu/A
ppData/Local/Temp/claude/C--Users-avidu-Projects-
muneem/02b9e9c9-3806-45ca-9b24-2ecc35a81efd/tasks/bfoo51v4e.output\"; for i in $(seq 1 90); do
s=$(wc -c < \"$f\" 2>/dev/null || echo 0); if [ \"$s\" -gt 5 ]; then cat \"$f\"; exit 0; fi;
sleep 2; done; echo \"TIMEOUT: repro script produced no output after 180s (likely hung)\")"}

```

25. user (2026-06-10T16:28:25.785Z)

```

before: size = 8192
before: rows = 5
calling backup_database(p, p, KEY) ...
Timeout (0:00:25)!
Thread 0x000043c0 (most recent call first):
  File "C:\\Users\\avidu\\Projects\\muneem\\src\\muneem\\db.py", line 300 in backup_database
  File "C:\\Users\\avidu\\AppData\\Local\\Temp\\muneem_backup_repro.py", line 22 in <module>
EXIT CODE: 1

```

26. assistant (2026-06-10T16:28:46.507Z)

```

import sys, faulthandler
sys.path.insert(0, r"C:\\Users\\avidu\\Projects\\muneem\\src")
from pathlib import Path
from muneem import db as dbmod

faulthandler.dump_traceback_later(15, exit=True) # simulate the user killing the hung command

```

```

KEY = "00112233445566778899aabbccddeeff00112233445566778899aabbccddeeff"
p = Path(r"C:\Users\avidu\AppData\Local\Temp\muneeem_repro_ledger\led.db")
p.parent.mkdir(parents=True, exist_ok=True)
for f in p.parent.glob("*"):
    f.unlink()

with dbmod.connect(p, key=KEY) as conn:
    conn.execute("CREATE TABLE t (id INTEGER PRIMARY KEY, amount REAL)")
    conn.executemany("INSERT INTO t (amount) VALUES (?)", [(float(i),) for i in range(5)])
print("before: size =", p.stat().st_size, flush=True)
print("calling backup_database(p, p, KEY) ...", flush=True)
dbmod.backup_database(p, p, KEY)
print("RESULT: returned without error", flush=True)

```

27. user (2026-06-10T16:28:50.274Z)

File created successfully at: C:\Users\avidu\AppData\Local\Temp\muneeem_backup_repro2.py (file state is current in your context ? no need to Read it back)

28. user (2026-06-10T16:29:15.344Z)

```

before: size = 8192
calling backup_database(p, p, KEY) ...
Timeout (0:00:15)!
Thread 0x00001dbc (most recent call first):
  File "C:\Users\avidu\Projects\muneeem\src\muneeem\db.py", line 300 in backup_database
  File "C:\Users\avidu\AppData\Local\Temp\muneeem_backup_repro2.py", line 19 in <module>
EXIT: 1
total 768
drwxr-xr-x 1 avidu 197609 0 Jun 10 21:59 .
drwxr-xr-x 1 avidu 197609 0 Jun 10 21:59 ..
-rw-r--r-- 1 avidu 197609 0 Jun 10 21:59 led.db

```

29. assistant (2026-06-10T16:29:40.305Z)

The repro is conclusive. Cleaning up the temp artifacts, then reporting.

30. user (2026-06-10T16:29:41.760Z)

(Bash completed with no output)

31. user (2026-06-10T16:30:00.138Z)

Structured output provided successfully

32. assistant (2026-06-10T16:30:07.693Z)

Verification complete. The finding is real (confirmed empirically): `backup\_database` unlinks the destination before any validation with no dest==source guard, so `muneeem db backup <ledger-path>` destroys the ledger. The only inaccuracy is the claimed exit-0 success ? the command actually hangs at `source.backup(target)` (db.py:300) and leaves a 0-byte file after the user kills it ? which doesn't change the data-loss outcome or the high severity. Structured output has been submitted.